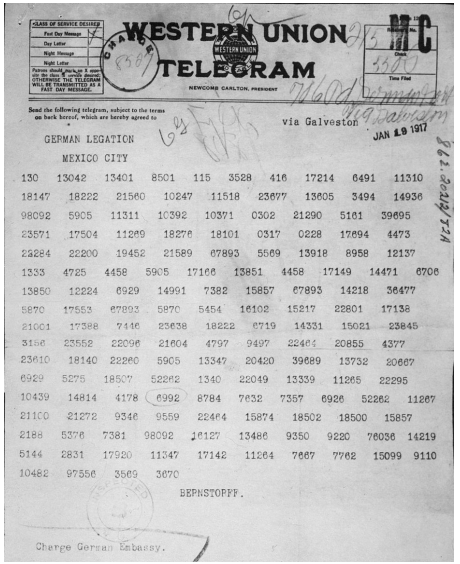
Cryptography Fundamentals

Cryptography

- Hiding information in plain sight!
- At its core is the aim to change ordered data into a seemingly random string
  - Using a secret key

$$C = F(P, k)$$

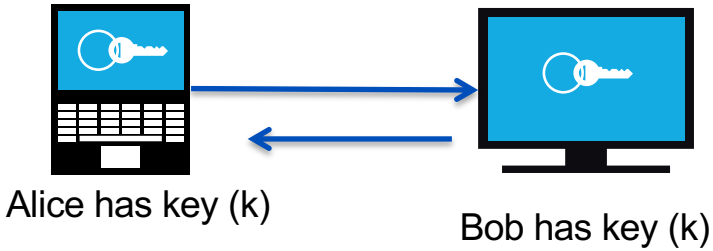
P – plain text
C – cipher text
k – cryptographic key



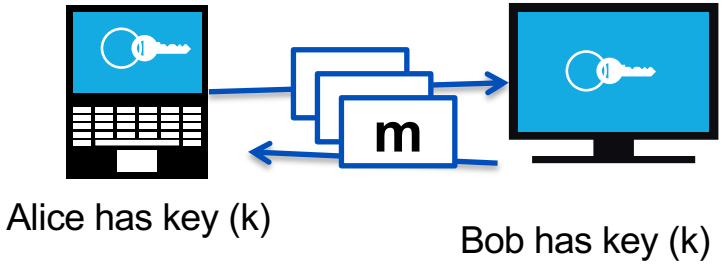
https://www.flickr.com/photos/35740357@N03

Crypto Core

- Secure key establishment



- Secure communication



Confidentiality and integrity

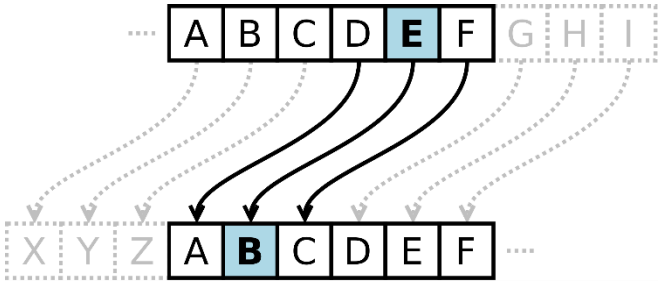
Early Ciphers – Substitution

- replacing an alphabet with another character of the same alphabet set

- Example:
  - Caesar cipher, a mono-alphabetic system in which each character is replaced by the third character in succession
  - Vigenere cipher, a poly-alphabetic cipher that uses a 26x26 table of characters

Early Ciphers – Substitution

- Caesar cipher



Plain	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
Cipher	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W

Plain Text: Learning cryptography is fun

Cipher Text: Ibxokfkd ???

https://en.wikipedia.org/wiki/Caesar_cipher

Early Ciphers – Substitution

- Vigenere cipher

Plain text:	L	e	a	r	n	i	n	g		c	r	y	p	t	o	g	r	a	p	h	y		i	s		f	u	n
Key:	A	P	N	I	C	A	P	N		I	C	A	P	N	I	C	A	P	N	I	C		A	P		N	I	C
Cipher Text:	L	t	n	z	p	i	c	t																				

Plain Text

Key

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
A	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
B	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A
C	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B
D	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C
E	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D
F	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E
G	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F
H	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G
I	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H
J	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I
K	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J
L	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K
M	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L
N	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M
O	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N
P	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
Q	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
R	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
S	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
T	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
U	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
V	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
W	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
X	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
Y	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X
Z	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y

https://en.wikipedia.org/wiki/Vigenère_cipher

Early Ciphers – Transposition

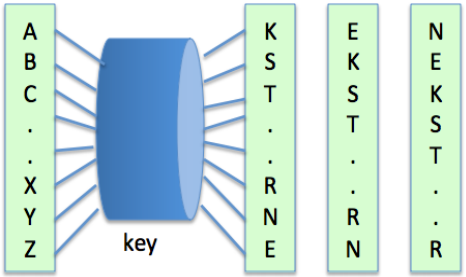
- No letters are replaced, they are just rearranged.

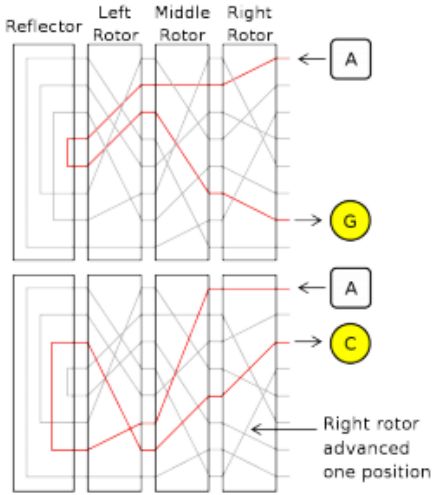
- Example:
  - Rail Fence Cipher – another kind of transposition cipher in which the words are spelled out as if they were a rail fence.

```
T...U...B...N...J...E...E...E...Y..
.H.Q.I.K.R.W.F.X.U.P.D.V.R.H.L.Z.D.G.
..E...C...O...O...M...O...T...A...O
```

Rotor Machines (1870-1943)

Hebern machine (single rotor)





Enigma machine (3-5 rotors)

Source: Wikipedia (image)

Modern Crypto Algorithms

- specifies the mathematical transformation that is performed on data to encrypt/decrypt
- Crypto algorithm is NOT proprietary
- Analyzed by public community to show that there are no serious weaknesses
- Explicitly designed for encryption

https://www.iso.org/standard/52906.html

Kerckhoff's Law (1883)

- The system must not be required to be secret, and it must be able to fall into the hands of the enemy without inconvenience.

The security of the system must rest entirely on the secrecy of the key, not the algorithm

Auguste Kerckhoffs

Work Factor

- The amount of processing power and time to break a crypto system
  - No system is unbreakable!

- The idea is to make it expensive

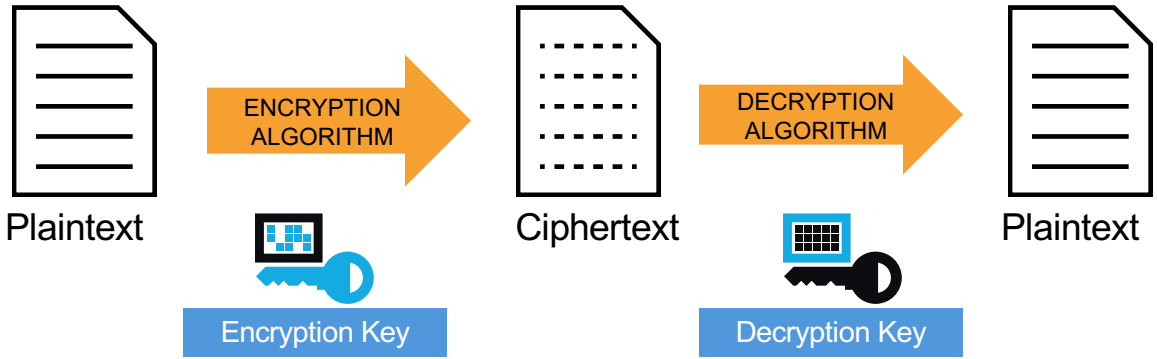
Properties of a Good Cryptosystem

- There should be no way short of enumerating all possible keys to find the key from any amount of ciphertext and plaintext, nor any way to produce plaintext from ciphertext without the key.
- Enumerating all possible keys must be infeasible.
- The ciphertext must be indistinguishable from true random values.

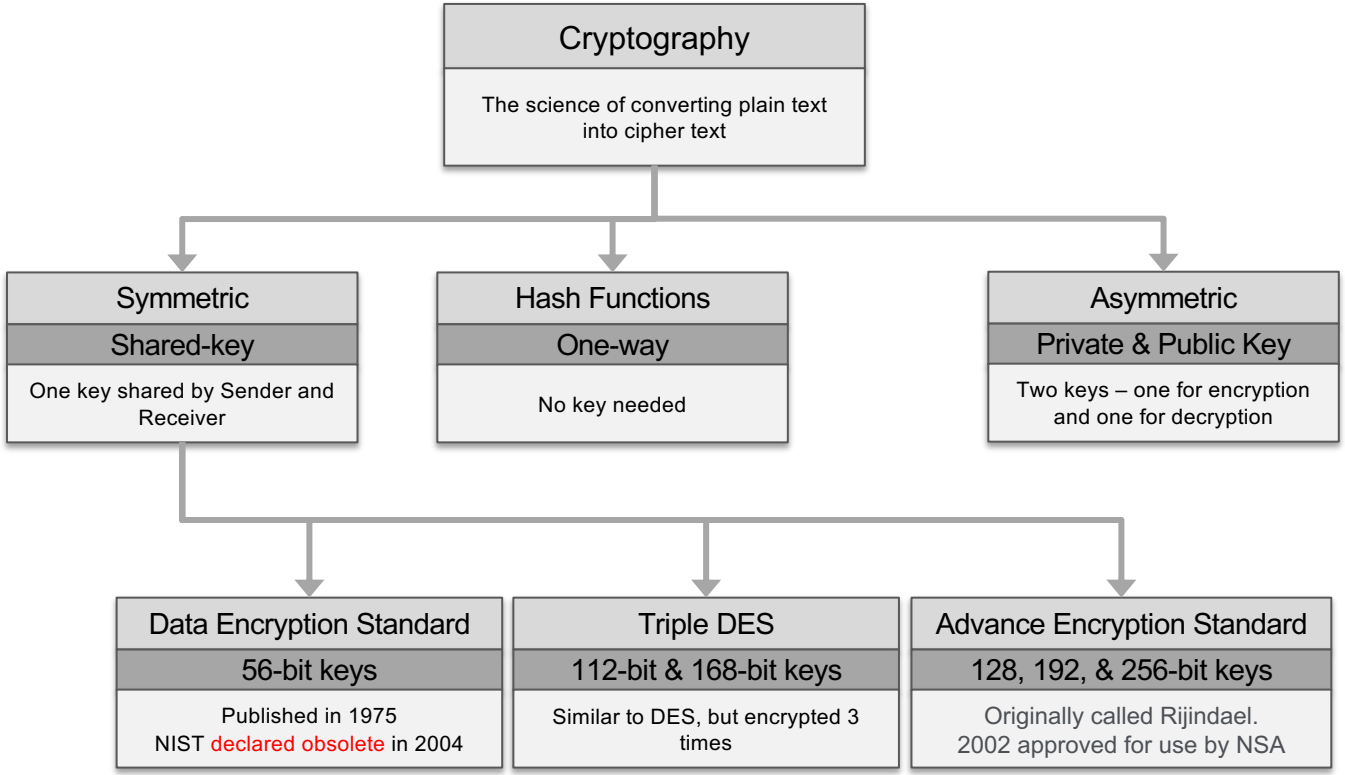
Encryption

- Transforming plaintext to ciphertext using a cryptographic key
- Used all around us
  - Application Layer – used in secure email, database sessions, and messaging
  - Session layer – using Secure Socket Layer (SSL) or Transport Layer Security (TLS)
  - Network Layer – using protocols such as IPsec

Encryption and Decryption



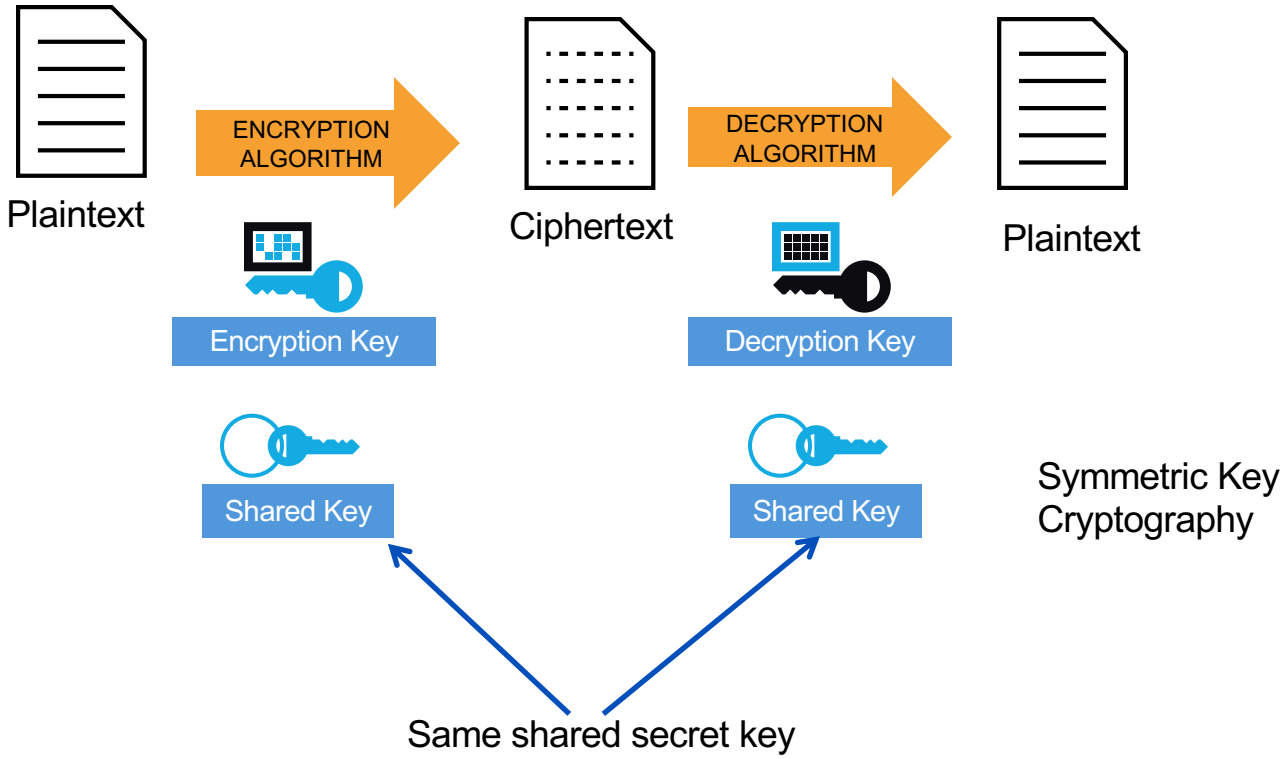
Crypto Overview



Symmetric Key Encryption

- Uses a single key to both encrypt and decrypt information
- Also known as a secret-key algorithm
  - The key must be kept a “secret” to maintain security
  - This key is also known as a private key
- Follows the more traditional form of cryptography with key lengths ranging from 40 to 256 bits.
- Examples:
  - DES, 3DES, AES, RC4, RC6, Blowfish

Symmetric Encryption



Symmetric Encryption

- Advantages
  - fast computation since the algorithms require small number of operations

- Disadvantages:
  - The sender and receiver needs to know the shared secret key before any encrypted conversation starts
    - How do we securely distribute the shared secret-key between the sender and receiver?
  - What if you want to communicate with multiple people, and each communication needs to be confidential?
    - How many keys do we have to manage? A key for each!
    - Key EXPLOSION!

Symmetric Key Algorithm

Symmetric Algorithm	Key Size
DES	56-bit keys
Triple DES (3DES)	112-bit and 168-bit keys
AES	128, 192, and 256-bit keys
IDEA	128-bit keys
RC2	40 and 64-bit keys
RC4	1 to 256-bit keys
RC5	0 to 2040-bit keys
RC6	128, 192, and 256-bit keys
Blowfish	32 to 448-bit keys

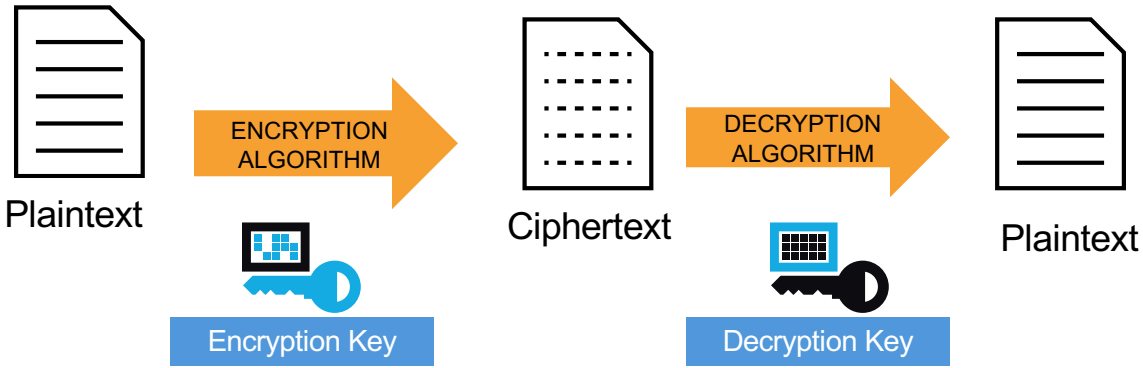
Note:
Longer keys are more difficult to crack, but more computationally expensive.

Data Encryption Standard (DES)

- Developed by IBM for the US government in 1973-1974, and approved in Nov 1976.
- Based on Horst Feistel's Lucifer cipher
- block cipher using shared key encryption, 56-bit key length
- Block size: 64 bits

DES: Illustration

64-bit blocks of input text



56-bit keys +
8 bits parity

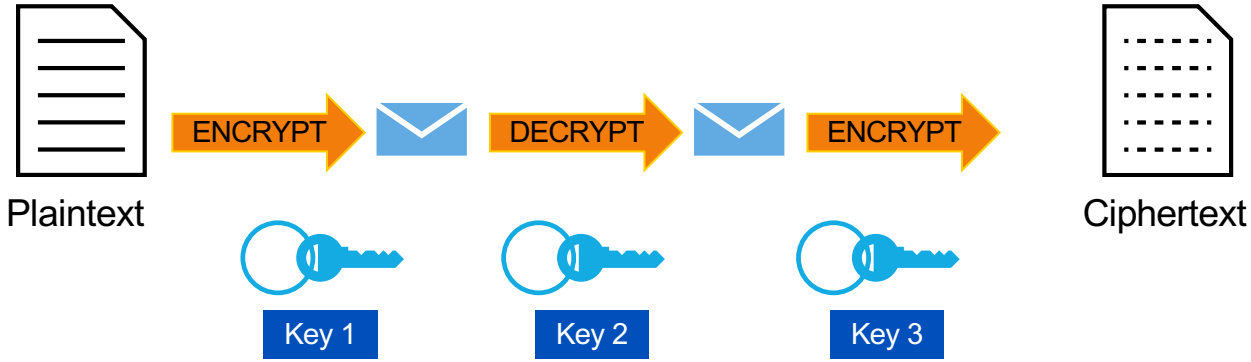
Triple DES

- 3DES (Triple DES) – a block cipher that applies DES three times to each data block
- Uses a key bundle comprising of three DES keys (K1, K2, K3), each with 56 bits excluding parity.
- DES encrypts with K1, decrypts with K2, then encrypts with K3

$$C_i = E_{K3}(D_{K2}(E_{K1}(P_i)))$$

- Disadvantage: very slow

3DES: Illustration



- Note:
  - If Key1 = Key2 = Key3, this is similar to DES
  - Usually, Key1 = Key3

Advanced Encryption Standard (AES)

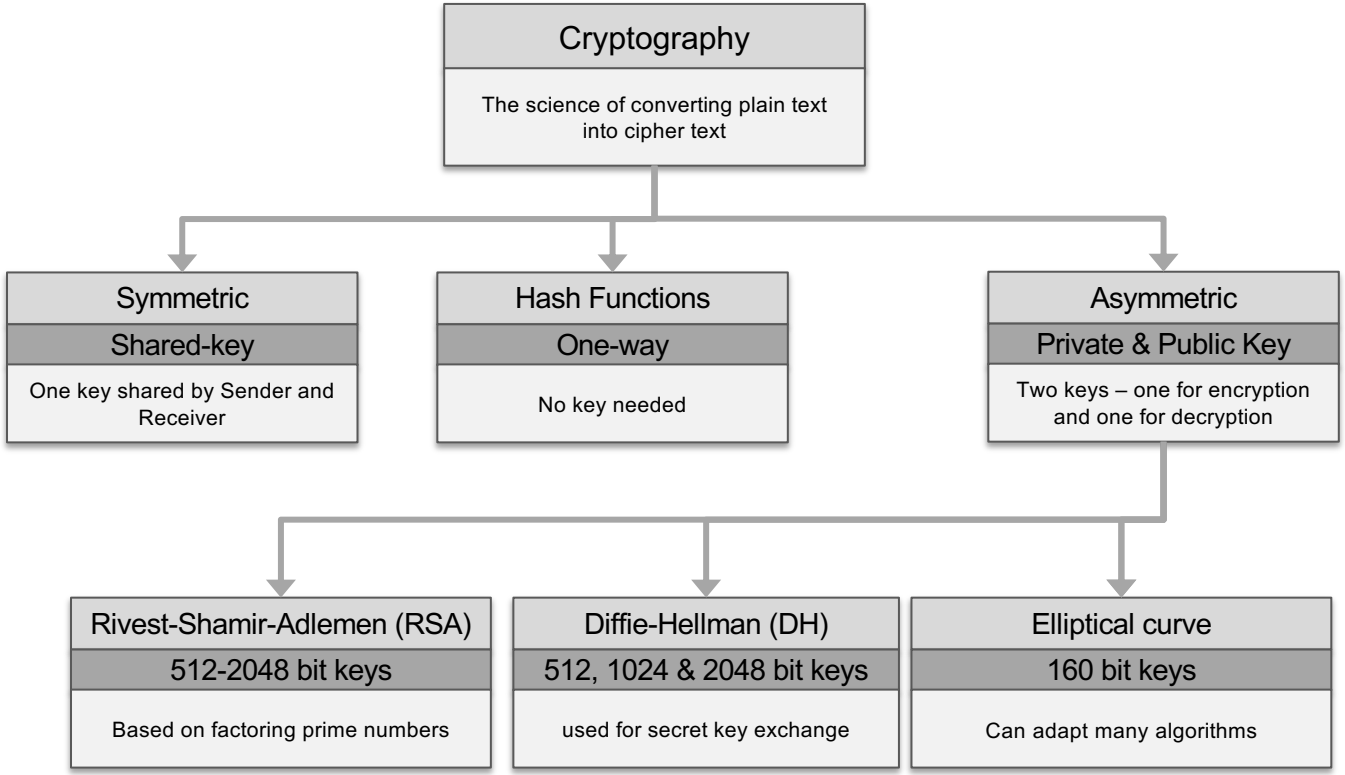
- Published in November 2001
- Symmetric block cipher
- Has a fixed block size of 128 bits
- Has a key size of 128, 192, or 256 bits
- Based on Rijndael cipher which was developed by Joan Daemen and Vincent Rijmen
- Better suited for high-throughput, low latency environments

Rivest Cipher

- Chosen for speed and variable-key length capabilities
- Designed mostly by Ronald Rivest
- Each of the algorithms have different uses

RC Algorithm	Description
RC2	Variable key-sized cipher used as a drop in replacement for DES
RC4	Variable key sized stream cipher; Often used in file encryption and secure communications (SSL)
RC5	Variable block size and variable key length; uses 64-bit block size; Fast, replacement for DES
RC6	Block cipher based on RC5, meets AES requirement

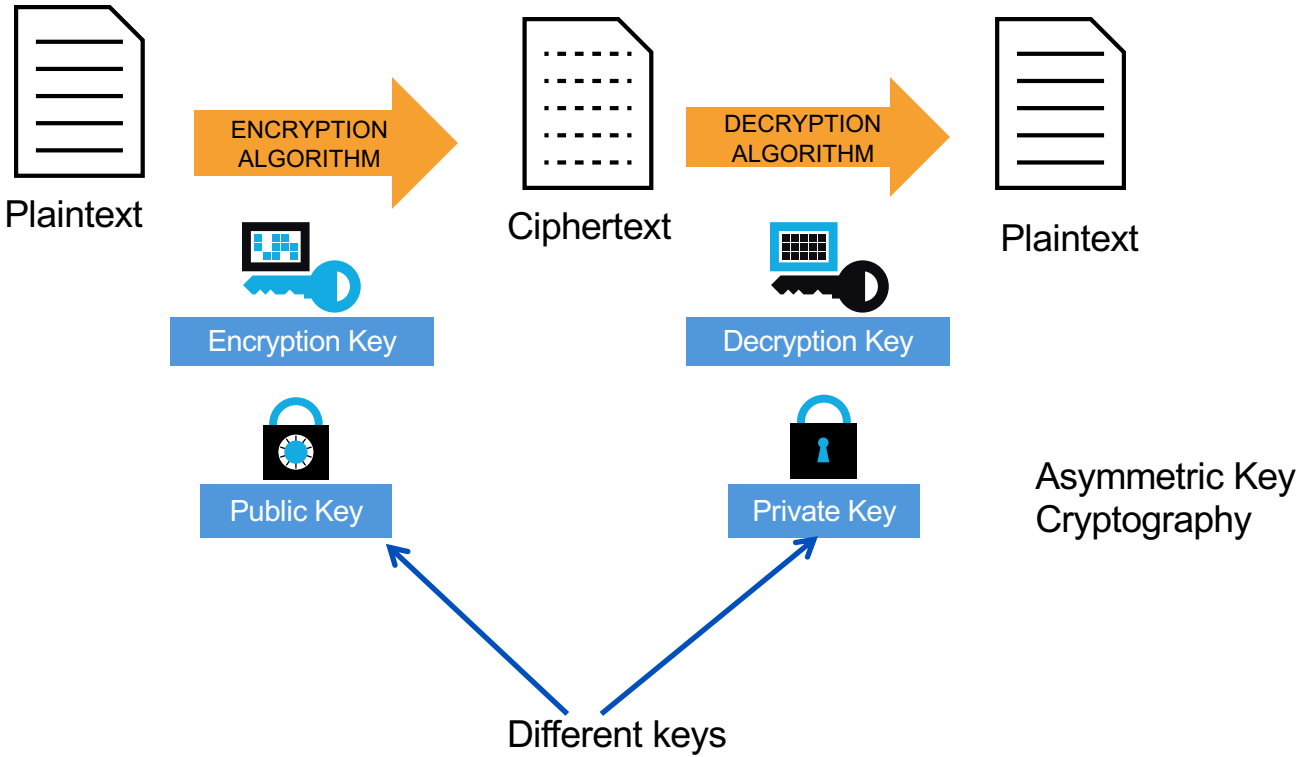
Crypto Overview



Asymmetric Key Encryption

- Also called public-key cryptography
  - Keep private key private
  - Anyone can see public key
- Public and private key pairs
  - The key pairs are mathematically linked
  - Messages encrypted with one key can only be decrypted by the other key of the key pair
- Examples:
  - RSA, DSA, Diffie-Hellman, ElGamal, PKCS

Asymmetric Encryption



Asymmetric Key Algorithms

Algorithm	Key Size (bits)	Description
RSA	512-2048	- Rivest-Shamir-Adlemen - Based on factoring 100 to 200 digit prime numbers - Base on the assumption that while it is easy to compute products of two large numbers, it is very difficult to factor a large number to be a product of two primes
DSA	512-1024	- Digital signature algorithm - Provides capability for authenticating messages
DH	512, 1024, 2048	- Diffie-Hellman - Allows two parties to agree on a key to encrypt messages (used for secret key exchange) - Security based on the assumption that while it is easy to raise a number to a certain power, it is difficult to find out which power was used
ElGamal	512-1024	- Based on DH key agreement - Used in GPG/PGP - Encrypted message becomes twice the size of the original (hence used only for sharing secret keys)
Elliptical curve	160	- Keys are much smaller - Can adapt many algorithms – DH or ElGamal

Asymmetric Encryption

- Advantages:
  - Solves the key explosion and distribution problem
  - No exchange of confidential information before communication
    - Public key is published (everyone knows)
    - Private key is kept secret (only the owner knows)

- Disadvantages
  - Much slower than symmetric algorithms

Diffie-Hellman key ‘exchange’

- DH algorithm
  - secure way to generate a shared secret between two parties
  - The key is NEVER exchanged or transmitted

DH key ‘exchange’

- Alice and Bob agree on two random primes (x and y)
- Alice and Bob pick a secret number each (a and b)
  - Which they DON’T share
- Alice computes $A = x^a \bmod y$ and sends to Bob
- Bob computes $B = x^b \bmod y$ and sends to Alice
- Alice then computes:

$$S = B^a \bmod y \Rightarrow = (x^b \bmod y)^a \bmod y = x^{ba} \bmod y$$

- Bob also computes:

$$S = A^b \bmod y \Rightarrow = (x^a \bmod y)^b \bmod y = x^{ab} \bmod y$$

DH in Colour ☺

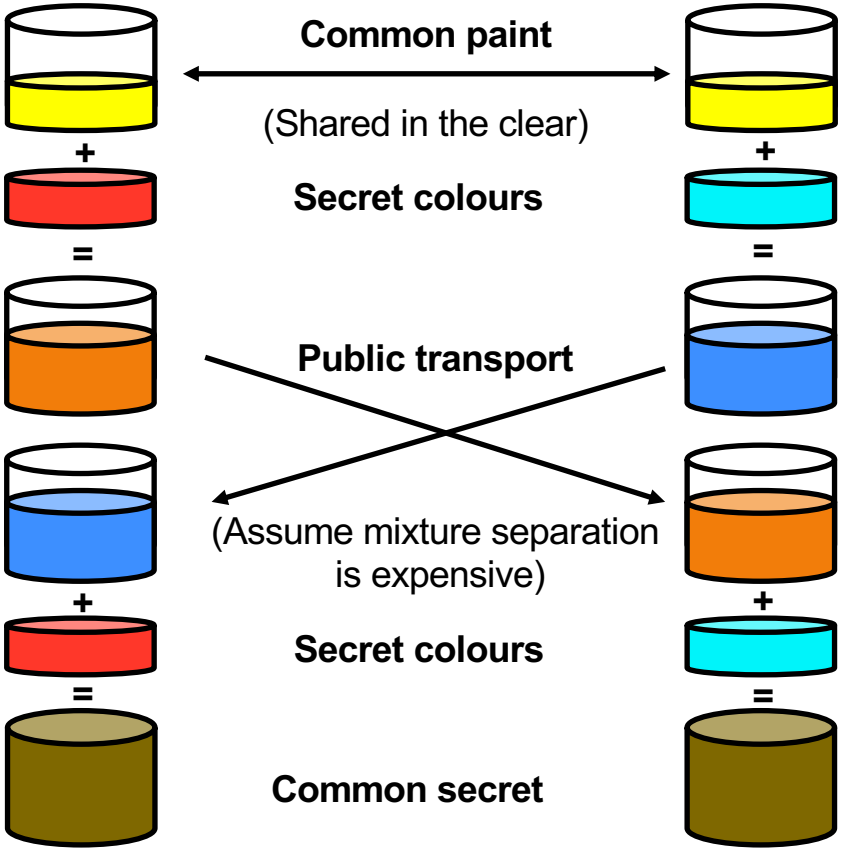


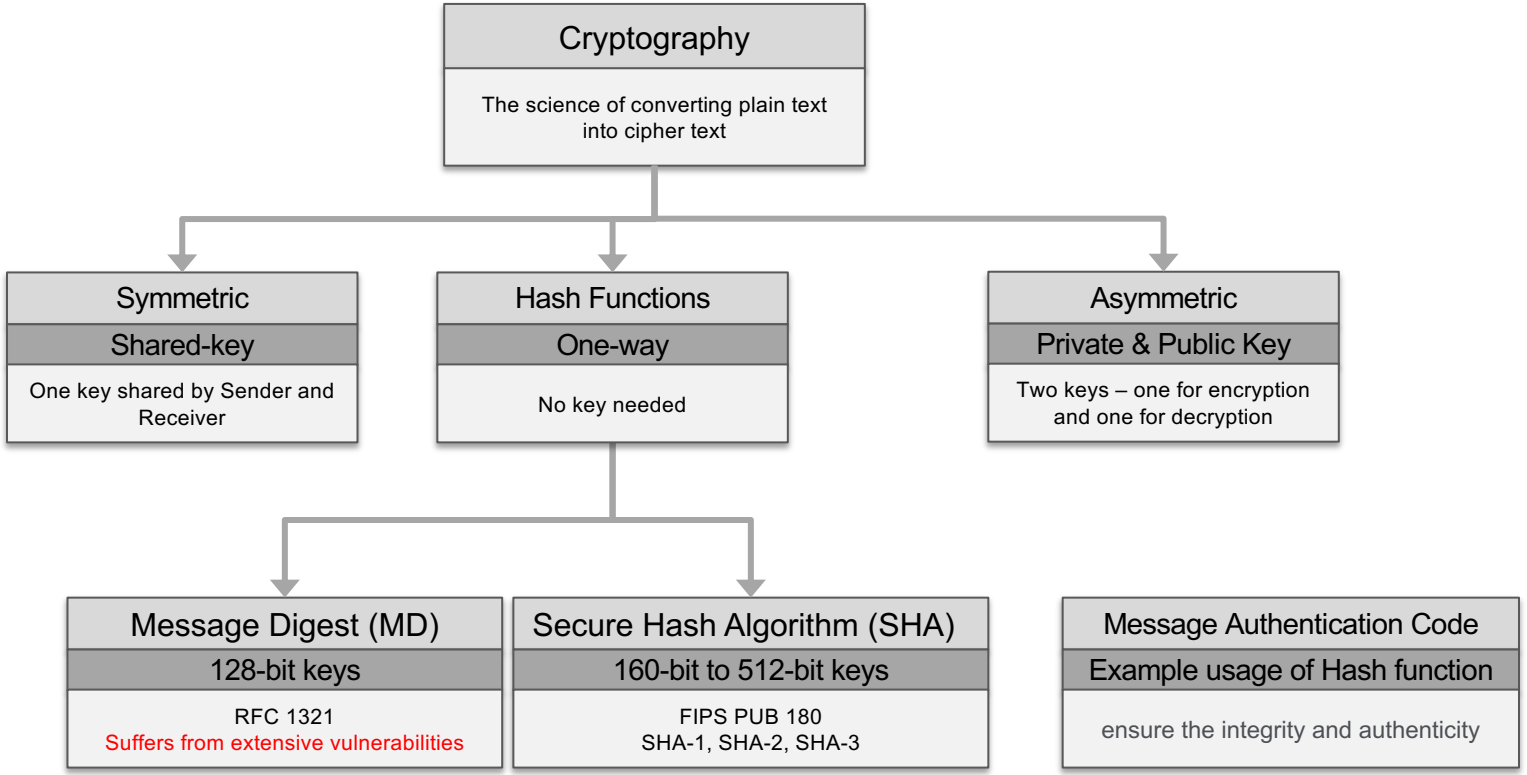
Image source:
https://en.wikipedia.org/wiki/Diffie%E2%80%93Hellman_key_exchange

Diffie-Hellman key ‘exchange’

- Without even knowing what secret each used, Alice and Bob generated the same result!
  - The shared-secret
  - Even if evil “Eve” is listening on the wire
    - Can see x, y, A, B
  - She cannot compute the same result since she would not know Bob and Alice’s secret

- Unlike normal exponentiation, which we can compute by $e^y = \log_e y$
- modular exponentiation or modular log(x) is difficult to compute!

Crypto Overview



Cryptographic Hashing

- Takes a message of arbitrary length and outputs a fixed-length output
  - called the hash or message digest, or digital fingerprint

- One-way mathematical function
  - Easy to compute, difficult to reverse

Arbitrary length data

Hash Function

Fixed-length hash value

Cryptographic Hashing

- A form of signature that uniquely represents a data
- Single bit change in input → large indeterminate change in output
- Example:

```
tashi@tashi-2 ~> echo TestHash1 | shasum -a 256
6b60ce02c9d3d811dfa0e5970ce7ec5ff11d2818d1a73718952e4afc2d65fb17  -
tashi@tashi-2 ~> echo TestHash2 | shasum -a 256
a85d8bea9956eccc0dc680e3f1682822682e4710f60d79e362a2007eb6854324  -
```

  - Helps protect integrity of data by identifying changes

Cryptographic Hashing

- Uses:
  - Verifying integrity
    - Ex: when downloading files from Internet, a hash is provided
  - Password Authentication
    - Hashes are commonly used for storing and verifying passwords
  - Digitally signing documents

Well-known Hash Functions

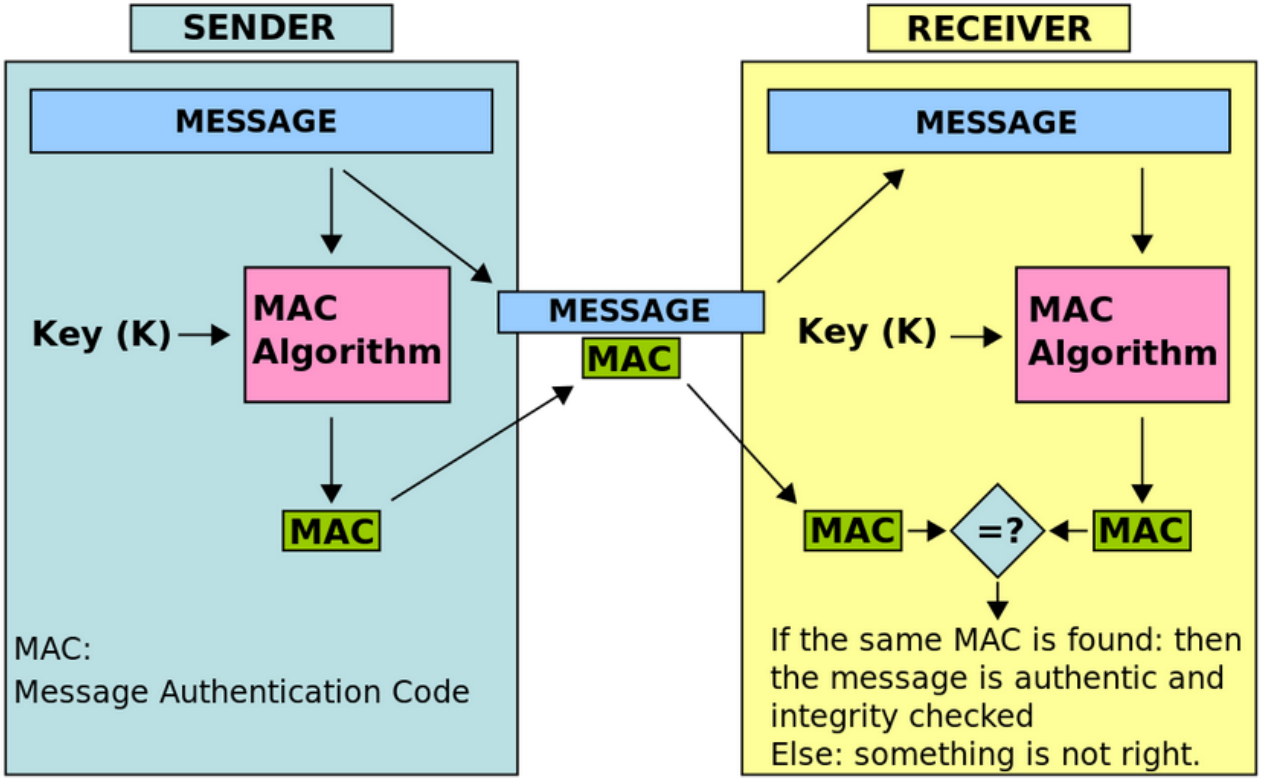
- Message Digest (MD) Algorithm
  - Outputs a 128-bit hash output (fingerprint)
    - 32-hex digits
  - MD5 is widely-used
    - Collisions found since 2013
    - DO NOT USE!

- Secure Hash Algorithm (SHA)
  - SHA-1 produces a 160-bit digest (40-hex digits)
    - Collisions found since 2017
    - Widely-used (TLS, SSL, PGP, SSH, S/MIME, IPsec) ☹
  - Use SHA-2 and SHA-3 (produce longer hash values)

Message Authentication Code

- Provides integrity and authenticity
- How it works:
  - In the sender side, the message is passed through a MAC algorithm to get a MAC (or Tag)
  - In the receiver side, the message is passed through the same algorithm
  - The output is compared with the received tag and should match
- Uses the same secret key
- Can also use hash function to generate the MAC, called Hash-based Message Authentication Code (HMAC)

Message Authentication Code

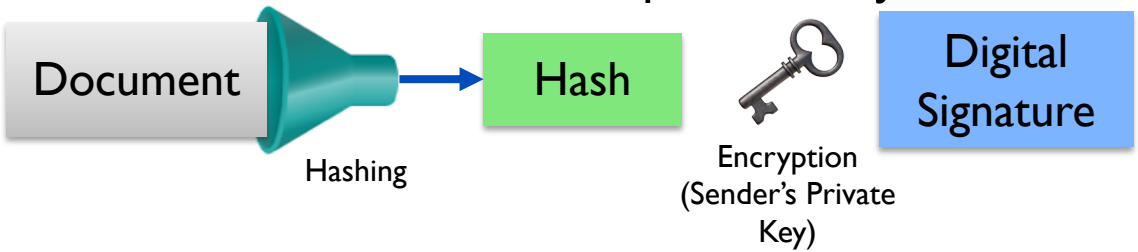


https://en.wikipedia.org/wiki/Message_authentication_code

Digital Signature

- Electronic documents can be signed
  - to prove the identity of the sender, and
  - the integrity of the message

- Encrypted hash of the message
  - Hash the data
  - Encrypt the hash with the sender's private key

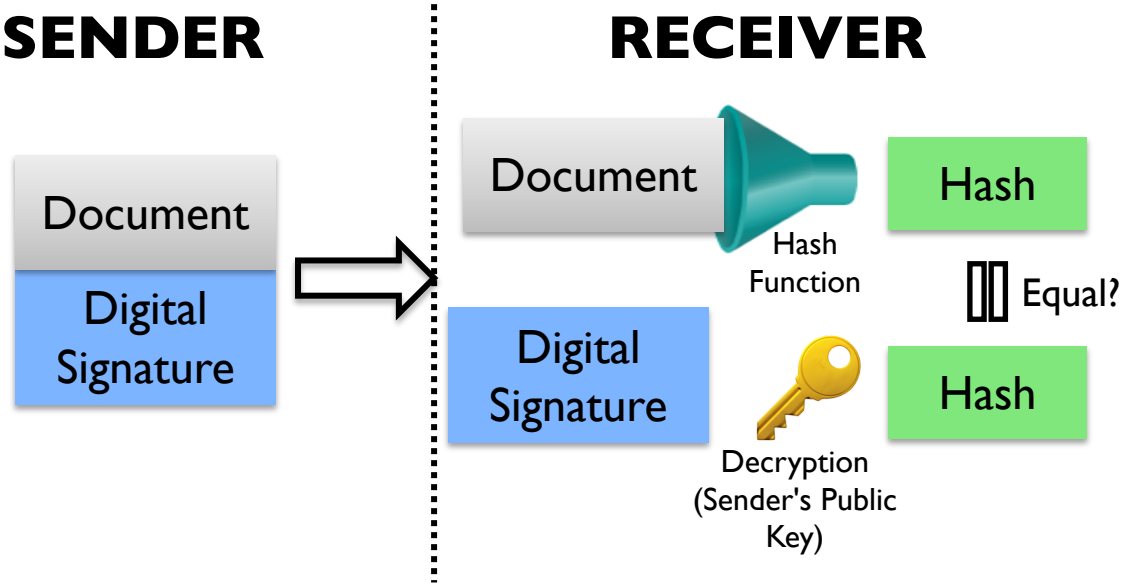


Digital Signature Validation

- Sender
  - Appends the signature to the original document
  - Sends to receiver

- Receiver
  - Computes the hash of the received data
    - Using same hash function
  - Decrypts the encrypted hash (signature) using sender's public key
    - Authentication
  - Compares the hashes
    - If match, the data was not modified (integrity) and signed by the sender

Digital Signature Validation



Example

```
-----BEGIN PGP SIGNED MESSAGE-----
Hash: SHA1

hello
-----BEGIN PGP SIGNATURE-----
Version: GnuPG v1

iQCVAwUBVsDzeDoo7EGKtqONAQKLFQQApJs+32OcZFZUdzQAO6GT8px6+F20CmO/
hInDACZnM2mvKP34J+fdslYyZWaivlhcaYeQsel+yvVjlO5NOLkdpjyOHw0ce99a
kAOP5cvSzw+fxGLkegrM3IhVCHlinKzLswpRCGOWP4xyAi1qaNPoykUzuInvnZ3u
3dVssbaXSN0=
=za1/
-----END PGP SIGNATURE-----
```

https://www.gpg4win.org (Windows)
https://www.gpgtools.org (OS X)

Attacks against Hashing

- Rainbow tables
  - Hashing algorithms create a fixed length output
  - What if you pre-compute hashes for all known inputs
    - Ex: passwords 1-8 characters long, with upper/lower case letters and numbers.
  - Then when you are presented with a hash, you can look it up to see what the plaintext input is
  - Now you have got a Rainbow Table!
    - ** Rainbow tables are more complex, but the above description is good enough
    - https://en.wikipedia.org/wiki/Rainbow_table

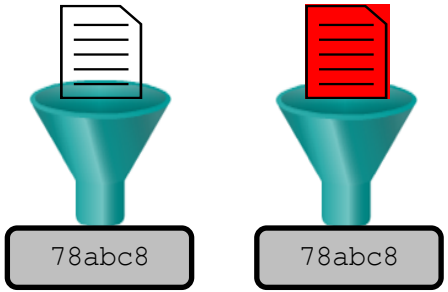
Attacks against Hashing

- Cryptographic salt
  - Things always taste better with salt ;-)

- Each bit of salt used doubles the amount of storage and computation required
  - Ex: If users create passwords from a dictionary of 100K words
  - Without salt
    - need to create a dictionary of 100K hashes to find a password match
  - With salt
    - Let's say 32-bit salt ~ salt possibilities = 2^32
    - Hash calculations need = 100K*2^32 = 429,496,729,600,000

Attacks against Hashing

- Hash collisions
  - fixed length output
  - With weaker algorithms/functions, two different inputs could produce the same output
    - Remember SHATTERED?
  - Ex:
    - $Hash(good\_doc) = 78abc8$
    - $Hash(malware\_virus) = 78abc8$
  - Do not use MD5, SHA-1




Questions

Thank You!

END OF SESSION